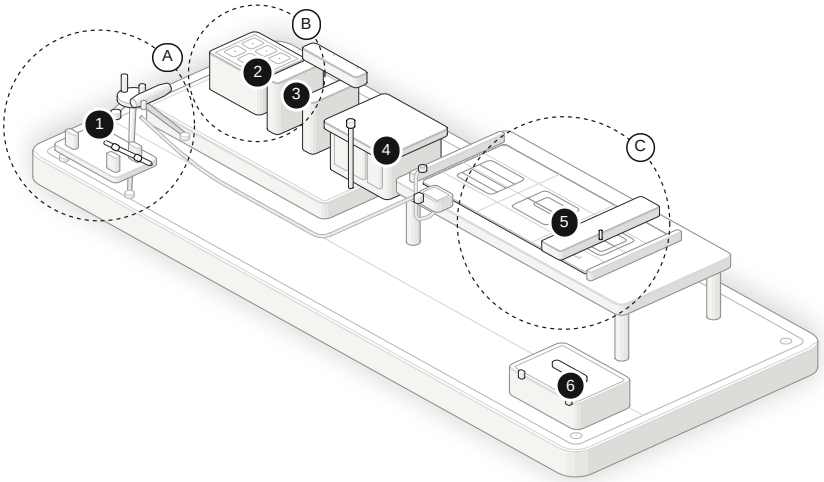
As-built



How does the OpenAPI document stay true to what the server serves?

ApiDescription crawls the live handler tree — concerns, then layout, then service methods — instead of a parallel description. The document is drawn from what actually answers, its schemas come from the same serializer options you serve with, and it is cached with a checksum and served content-negotiated as JSON or YAML.

5 explorers
CONCERN, LAYOUT, SERVICE, COLLECTION, METHOD

1 source
THE LIVE HANDLER GRAPH, NEVER A PARALLEL MODEL

2 formats
JSON OR YAML, BY CONTENT NEGOTIATION

1 line
.ADDOPENAPI() PUTS IT ON YOUR HOST

0 drift
WHAT ANSWERS IS WHAT IS DOCUMENTED

PARTS

01 **The theodolite**
ApiDescription, sighted on the live tree — concerns, layout, services.

02 **Machine A**
An endpoint the survey has sighted: its drawing joins the sheet.

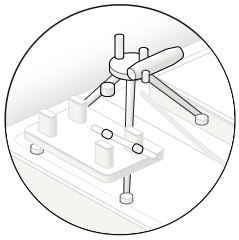
03 **Machine B**
The explorer chain decides, then descends, handler by handler.

04 **Machine C**
Service methods fan out to one OpenApiOperation per verb.

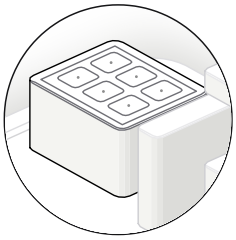
05 **The drafting table**
The document, drawn at full scale from the measurements.

06 **The instrument case**
The cache: the finished document, kept with a checksum.

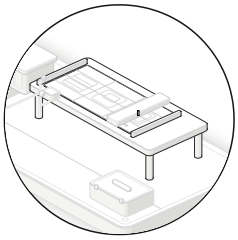
CLOSE-UPS



(A) **The instrument**
One OpenApiOperation per verb at the leaf; XML doc comments supply summaries, attributes layer the rest.

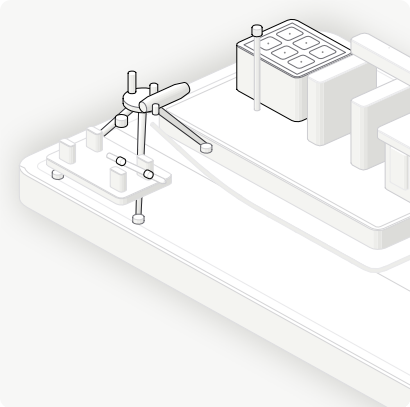


(B) **A machine**
Whatever the tree serves: a webservice, a controller, a nested layout — if it answers, it gets surveyed.



(C) **The sheet**
Swagger UI, Redoc and Scalar can host it straight from Modules/ApiBrowsing — the drawing and the machine never disagree.

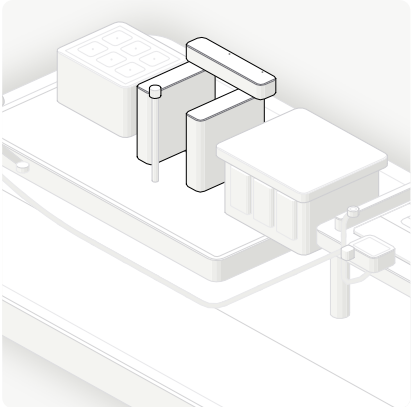
How it works



01

Crawl, don't describe

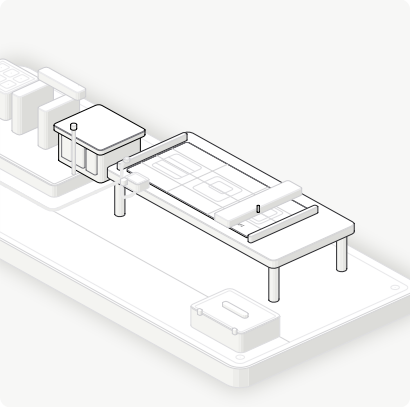
ApiDescription intercepts openapi.json and walks the live tree. No parallel description exists to fall out of sync.



02

The explorer chain

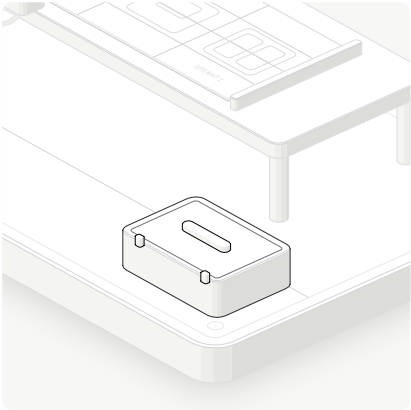
Each explorer decides whether it can read a handler, then descends: concerns, layout roots, services, method collections, methods.



03

The sheet is drawn

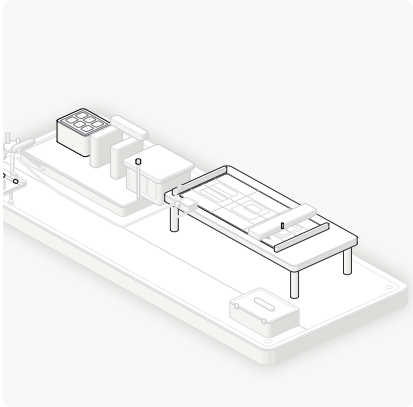
Arguments become parameters, content types become schemas — built from the same serializer options the responses use.



04

Sighted before, served from cache

A re-sight hits the checksummed cache: same document, content-negotiated JSON or YAML with Vary: Accept.



05

Security on the sheet

Authentication concerns document their own schemes, and [RequireRole] is where the 401 and 403 responses come from.

WHY CRAWL INSTEAD OF DESCRIBE?

A parallel description drifts the day someone changes a route and forgets the spec. Crawling the real object graph means the document cannot disagree with the server: the operations it lists are the ones the runtime dispatches, and the schemas describe the bytes on the wire.

SPECIFICATION

Chain	Concern, Layout, Service, Collection, Method	Negotiated	Vary: Accept — JSON or YAML
Source	the live handler tree itself	Security	.Security(...); auth concerns self-document
Schemas	NJsonSchema on your serializer options	Metadata	method > interceptors > class > concerns
Cache	checksummed via OpenApiConcernBuilder		